

Topic 7 – Self-Correcting Agentic Auditor

Advanced Planning, Reasoning & Self-Correction — Assessment Documentation

1. Solution Overview

The Self-Correcting Agentic Auditor is an n8n workflow that processes deterministic mock transaction records, validates them with a deterministic Code Tool, uses a Google Gemini-powered AI Agent to reason about safe corrections, re-validates corrected records, produces strict structured JSON, and routes each transaction to VALID or INVALID outputs.

2. Architecture

Node	Type	Purpose
Run Audit	Manual Trigger	Starts the audit
Mock Transaction Data	Code	Emits 8 deterministic transactions
Process One At A Time	Split In Batches	Sequential processing
Transaction Auditor Agent	AI Agent	Audits, corrects and re-validates
Chat Model	Google Gemini	LLM reasoning engine
validate_transaction	Code Tool	Deterministic validation
Structured Output Parser	Structured Output	Strict JSON schema
Validation Router	IF	VALID vs INVALID routing
Valid / Invalid Output	Set	Final branch outputs
Rate Limit Pacing	Wait	20-second Gemini pacing

3. AI Agent Configuration

Google Gemini is the Chat Model. Temperature is 0.0, maximum output tokens are 2048, and the AI Agent maximum iteration setting is 4. The system prompt requires tool-based validation, safe minimal correction, re-validation, and refusal to fabricate missing or unsafe values.

4. Deterministic Validation Tool

The validate_transaction Code Tool does not use an LLM. It checks required fields, positive amount, allowed currency, transaction type, status, and YYYY-MM-DD date format. Its result is the authority for transaction validity.

5. Structured Output Schema

```
transaction_id: string
status: VALID | INVALID
original_transaction: object
corrected_transaction: object
validation_errors: string[]
corrections_made: string[]
iterations: number
validation_passed: boolean
```

The parser uses additionalProperties: false.

6. Mock Test Dataset

ID	Case	Expected
TXN-001	Fully valid	VALID immediately

ID	Case	Expected
TXN-002	Missing customer_id	INVALID / uncorrectable
TXN-003	currency = usd	VALID after USD correction
TXN-004	Negative amount	INVALID / uncorrectable
TXN-005	status = done	VALID after completed correction
TXN-006	Date 09/06/2026	VALID after date reformat
TXN-007	eur + Completed casing	VALID after correction
TXN-008	Missing amount + quantum_teleport	INVALID / uncorrectable

7. Live Validation Evidence — Execution #148

A controlled live execution using the real Gemini credential successfully demonstrated the agentic self-correction behavior. Three representative transactions were live-tested. The complete 8-transaction dataset remains configured, but the other five cases were not claimed as live-tested.

Transaction	Observed behavior	Final
TXN-003	usd → USD; re-validation passed; 2 iterations	VALID
TXN-005	done → completed; re-validation passed; 2 iterations	VALID
TXN-008	Missing amount + invalid type; no unsafe fabrication; 1 iteration	INVALID

8. Self-Correction Logic

The agent follows validate → inspect errors → apply safe minimal correction → re-validate. Safe examples include currency/status normalization and unambiguous date reformatting. Unsafe examples include inventing missing customer IDs or amounts, guessing a negative amount, or mapping an unknown transaction type. The agent stops when validation succeeds or the four-iteration maximum is reached.

9. Conditional Routing

The Validation Router evaluates the structured status field. VALID records go to Valid Transaction Output; INVALID records go to Invalid Transaction Output. Both branches emit clean JSON.

10. Quota-Resilience Design

Transactions are processed one at a time with a 20-second Wait between transactions. This pacing was added to respect the Gemini free-tier request limit and enabled the controlled live test to complete successfully.

11. Submission Evidence

- Exported n8n workflow JSON
- Loom demo: <https://www.loom.com/share/f099a3e01bd44d569f1b0b4fcb9b1e2e>
- Live execution evidence: Execution #148
- Full 8-case deterministic test dataset
- Validation/self-correction results
- GitHub repository to be created

12. Conclusion

The workflow implements deterministic validation, Gemini-based reasoning, iterative self-correction, strict structured output, and conditional VALID/INVALID routing. The critical correction and safe-refusal behaviors were demonstrated live in Execution #148.